

LEETCODE 127 · CS4137 GROUP 8

WORD LADDER

Shortest one-letter transformation between two words

hit

hot

dot

dog

cog

Ayal Yakobe (amy2127) · Leen Alshorafa (laa2202) · Aashir Khan (ak5445)

LeetCode 127 · Word Ladder

leetcode.com/problems/word-ladder

Given `beginWord`, `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence — each step changes exactly one letter, and every intermediate word must appear in the list. Return 0 if none exists.



Required signature

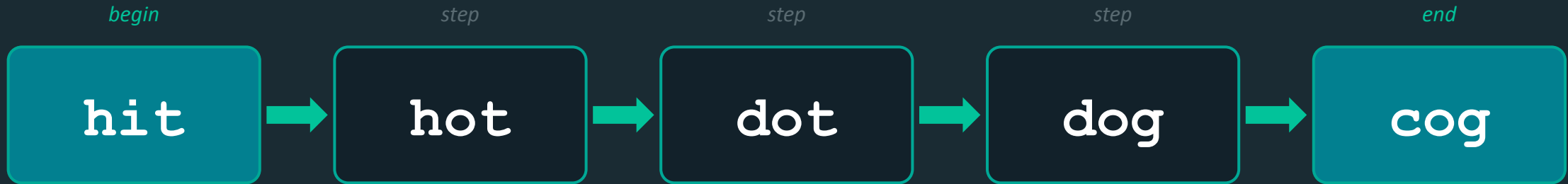
```
class Solution {  
    public int ladderLength(String beginWord,  
        String endWord, List<String> wordList) { ... }  
}
```

CONSTRAINTS

- $1 \leq \text{word length} \leq 10$
- `endWord.length == beginWord.length`
- $1 \leq \text{wordList.length} \leq 5000$
- All words lowercase & unique
- `beginWord` $\neq$ `endWord`

Change one letter at a time

Each step swaps a single letter, and every word you land on must be a real dictionary word. Count the words in the shortest such chain.



Answer = 5 words (the chain has 5 entries, counting both ends)

Key idea: words are nodes, one-letter swaps are edges → this is a shortest-path problem on an unweighted graph → **BFS finds it.**

What goes in, what comes out

IN Input

- `beginWord`
- `endWord`
- `wordList` (the dictionary)

File format (inputN.txt):

```
beginWord endWord
N
<word 1>
<word 2>
...
<word N>
```

→ Output

A single integer — the number of words in the shortest sequence, counting both endpoints.

0

returned when endWord is unreachable

Example: `hit → cog` outputs `5`

Three solutions, increasing in speed

We built a naive baseline, then optimized twice — each version is standalone and LeetCode-submittable.

1 Brute Force

Compare every pair of words to build edges, then plain BFS.

graph build

$$O(N^2 \cdot L)$$

2 Wildcard BFS

Bucket words by wildcard patterns ($h*t$). Neighbors share a bucket.

graph build

$$O(N \cdot L^2)$$

3 Bidirectional BFS

Same buckets, but search from both ends; expand the smaller frontier.

graph build

$$\approx O(k^{(d/2)})$$

Brute Force: compare every pair

Build the graph by checking **every pair** of words: if they differ by exactly one letter, add an edge. Then run a plain level-order BFS from beginWord.

Graph construction

$O(N^2 \cdot L)$

BFS traversal

$O(N^2)$ worst case

Verdict

simple, correct, slow

```
// the bottleneck:  $O(N^2)$  pairwise
for (int i = 0; i < n; i++)
    for (int j = i+1; j < n; j++)
        if (differsByOneLetter(
            nodes.get(i),
            nodes.get(j))) {
            adjacency.get(i).add(j);
            adjacency.get(j).add(i);
        }
// ...then standard BFS by level
```

Intentionally the "first attempt" — correct, but slows down sharply as wordList grows.

The pairwise scan doesn't scale

THE PROBLEM

With up to 5000 words, comparing every word to every other is **25,000,000** comparisons — before BFS even starts.

We don't actually need to test each pair directly.

THE INSIGHT

Words one letter apart share a **wildcard pattern**.

```
hot → *ot , h*t , ho*  
hat → *at , h*t , ha*  
      ^ shared bucket!
```

Group by pattern once → neighbors are free lookups.
No pairwise scan.

Wildcard-Pattern BFS

Each word emits L patterns into a `HashMap<String, List<String>>`. Two words are neighbors iff they share a bucket. BFS expands through buckets, not the whole list.

patternMap

"*ot" → hot, dot, lot

"h*t" → hot, hat, hit

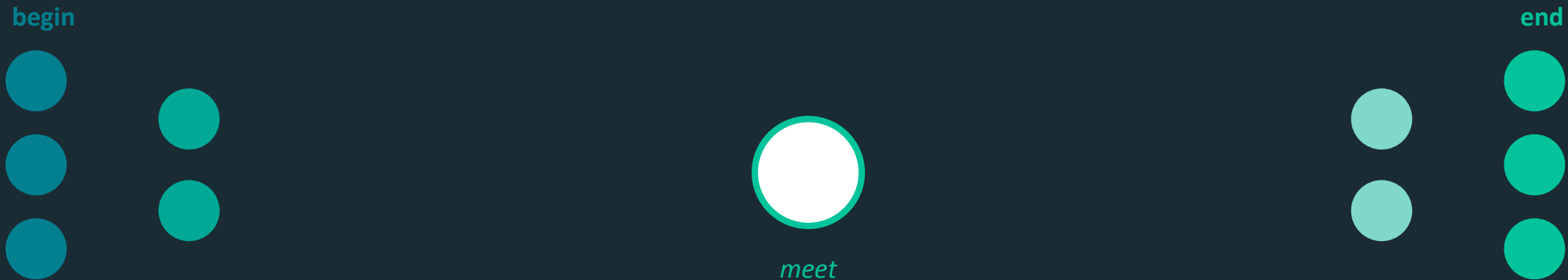
"do*" → dot, dog

```
// build buckets once: O(N·L²)
for (String w : wordList)
    for (int j=0; j<L; j++) {
        String p = w.substring(0,j)
            + '*' + w.substring(j+1);
        map.computeIfAbsent(p,
            k->new ArrayList<>())
            .add(w);
    }
// BFS then hops bucket→bucket
```

The standard accepted approach — dramatically faster on large word lists.

Bidirectional Wildcard BFS

Same buckets — but search from beginWord and endWord at once, always expanding the smaller frontier. Stop the moment the two searches meet.



Why it wins: two shallow trees beat one deep one — roughly $O(k^{(d/2)})$ vs $O(k^d)$. Biggest payoff on deep, heavily-branched graphs; on thin/linear graphs the gain over Attempt 2 is modest.

The meeting-point stitch

The heart of the final solution: expand the smaller queue, and detect when a neighbor is already known on the other side.

```
// pick the smaller frontier each round
if (queueBegin.size() <= queueEnd.size())
    result = expandOneLevel(queueBegin,
        distFromBegin, distFromEnd, ...);

// inside expandOneLevel, on each neighbor:
if (distOther.containsKey(next))
    // the searches met - stitch halves
    return distance + distOther.get(next);
if (!distOwn.containsKey(next)) {
    distOwn.put(next, distance + 1);
    queue.offer(next);
}
```

Smaller frontier first

keeps both search trees shallow and narrow

distOther lookup

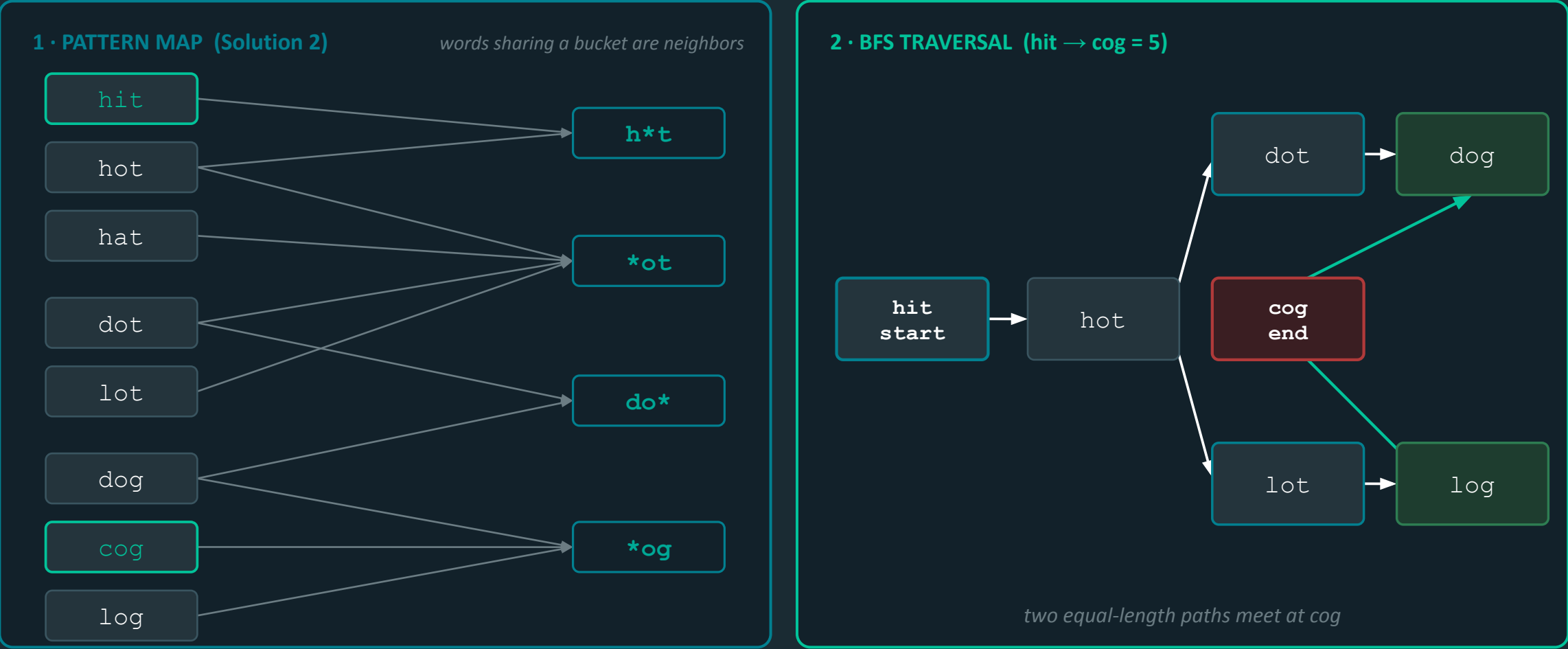
an O(1) check that the other side already reached this word

Stitch the halves

distance on this side + distance on the other = full ladder

From wildcard buckets to the search

Left: words sharing a pattern are neighbors. Right: BFS walks those edges from begin to end.



The pattern map (left) is how each word finds its neighbors in $O(1)$; BFS (right) then walks those edges level by level — `hit-hot-dot-dog-cog` = 5 words.

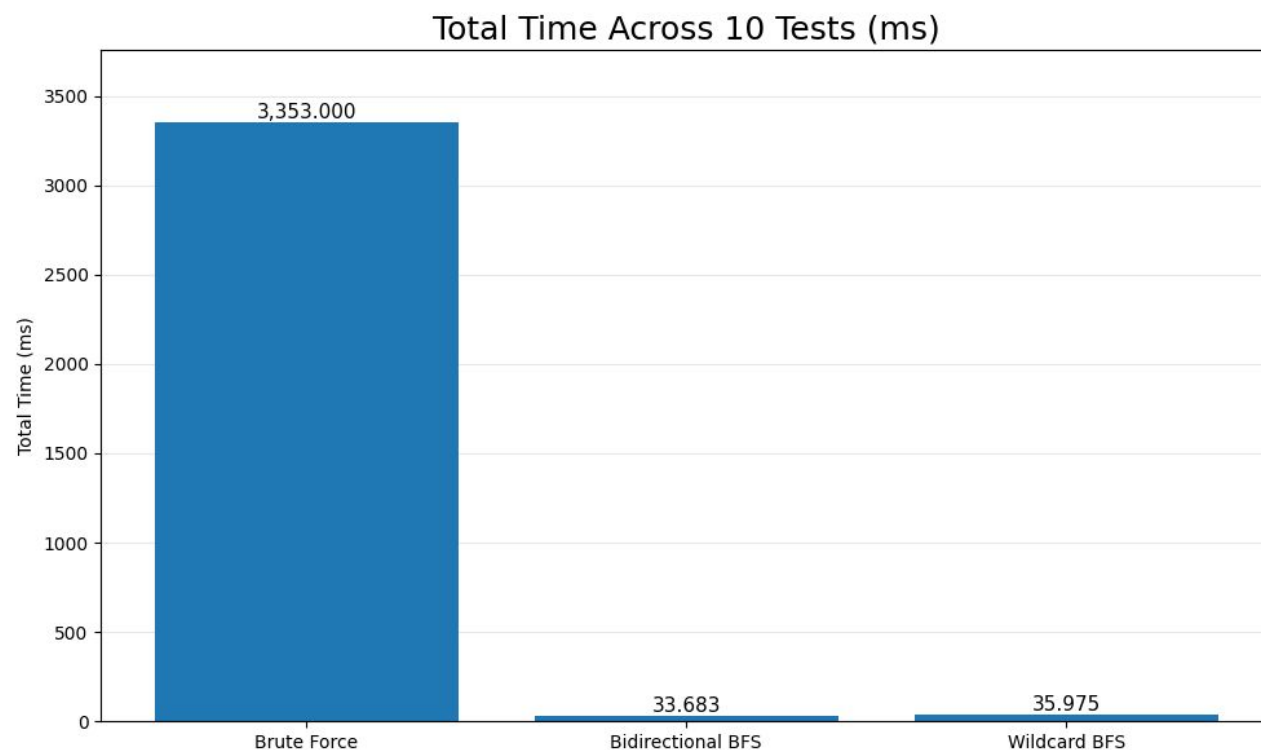
TEST CASES

10 inputs, each covering a case

#	What it covers	Out
1	Classic branching graph, multiple shortest paths	5
2	endWord missing from wordList	0
3	Minimal: single adjacent word	2
4	endWord present but disconnected	0
5	Hand-built 9-word chain, no shortcuts	10
6	Diamond graph, two equal-length paths	3
7	beginWord redundantly in wordList	3
8	5-letter words + unrelated distractor	3
9	Mid-size random (157 words), embedded chain	8
10	Stress test (3515 words), 15-step chain	16

All 30 runs pass — and get faster

Every solution × test-case combination passes. Average time per test drops with each optimization.



PASS RATE

30 / 30

BruteForce

335 ms/test

WildcardBFS

3,683 μ s/test

BidirectionalBFS

3,597 μ s/test

Timings vary with JIT warm-up / JVM state.

JProfiler execution-time breakdown: Hot Spots, Call Tree

Thread status: Runnable Thread selection: All thread groups Aggregation level: Methods Hot spot options: Self times

Hot Spot	Self Time	Average Time	Invocations
BruteForceSolution.differsByOneLetter	1,343 ms (38 %)	0 µs	6,191,861
java.lang.String.charAt	728 ms (20 %)	0 µs	25,759,504
20.9% - 728 ms - 25,759,504 hot spot inv. BruteForceSolution.differsByOneLetter			
20.9% - 728 ms - 25,759,504 hot spot inv. BruteForceSolution.ladderLength			
20.9% - 728 ms - 25,759,504 hot spot inv. TestRunner.main			
BruteForceSolution.ladderLength	579 ms (16 %)	57,977 µs	10
16.7% - 579 ms - 10 hot spot inv. TestRunner.main			
java.lang.String.length	353 ms (10 %)	0 µs	12,879,821
10.2% - 353 ms - 12,879,803 hot spot inv. BruteForceSolution.differsByOneLetter			
0.0% - 2 µs - 9 hot spot inv. BidirectionalBFSSolution.ladderLength			
0.0% - 2 µs - 9 hot spot inv. WildcardBFSSolution.ladderLength			
java.util.List.get	346 ms (9 %)	0 µs	12,383,872
9.9% - 346 ms - 12,383,872 hot spot inv. BruteForceSolution.ladderLength			
9.9% - 346 ms - 12,383,872 hot spot inv. TestRunner.main			
java.io.PrintStream.printf	21,514 µs (0 %)	614 µs	35
WildcardBFSSolution.buildPatternMap	16,548 µs (0 %)	1,838 µs	9
BidirectionalBFSSolution.buildPatternMap	14,780 µs (0 %)	1,642 µs	9
java.util.Map.computeIfAbsent	12,109 µs (0 %)	0 µs	72,068
java.io.BufferedReader.readLine	8,368 µs (0 %)	2 µs	3,735
java.util.List.add	5,605 µs (0 %)	0 µs	83,301
TestRunner.loadTestCases	5,522 µs (0 %)	5,522 µs	1
java.lang.String.substring(int)	4,981 µs (0 %)	0 µs	72,598
java.lang.String.substring(int, int)	4,558 µs (0 %)	0 µs	72,598
java.lang.String.matches	4,501 µs (0 %)	225 µs	20
TestRunner.main	4,138 µs (0 %)	4,138 µs	1
TestRunner.parseTestCase	2,665 µs (0 %)	266 µs	10
java.util.ArrayList.<init>()	2,560 µs (0 %)	0 µs	75,706
WildcardBFSSolution.lambda\$buildPatternMap\$0	2,432 µs (0 %)	0 µs	35,994
BidirectionalBFSSolution.lambda\$buildPatternMap\$0	2,417 µs (0 %)	0 µs	35,994
java.io.FileReader.<init>	2,148 µs (0 %)	107 µs	20
java.lang.String.replaceAll	1,921 µs (0 %)	36 µs	52
java.util.HashSet.<init>(java.util.Collection)	1,178 µs (0 %)	58 µs	20
BidirectionalBFSSolution.expandOneLevel	1,069 µs (0 %)	23 µs	45
java.util.HashSet.<init>	878 µs (0 %)	0 µs	12,465

Thread status: Runnable Thread selection: All thread groups Aggregation level: Methods

100.0% - 3,479 ms - 1 inv. TestRunner.main
96.4% - 3,353 ms - 10 inv. BruteForceSolution.ladderLength
69.7% - 2,425 ms - 6,191,861 inv. BruteForceSolution.differsByOneLetter
20.9% - 728 ms - 25,759,504 inv. java.lang.String.charAt
10.2% - 353 ms - 12,879,803 inv. java.lang.String.length
9.9% - 346 ms - 12,383,872 inv. java.util.List.get
0.0% - 574 µs - 7,518 inv. java.util.List.add
0.0% - 458 µs - 7,399 inv. java.lang.String.equals
0.0% - 239 µs - 3,717 inv. java.util.ArrayList.<init>()
0.0% - 235 µs - 3,849 inv. java.util.Iterator.hasNext
0.0% - 228 µs - 3,792 inv. java.util.Iterator.next
0.0% - 94 µs - 10 inv. java.util.List.contains
0.0% - 78 µs - 56 inv. java.util.Queue.offer
0.0% - 67 µs - 158 inv. java.lang.Integer.valueOf
0.0% - 28 µs - 56 inv. java.util.Queue.poll
0.0% - 27 µs - 54 inv. java.util.Queue.isEmpty
0.0% - 25 µs - 148 inv. java.lang.Integer.intValue
0.0% - 21 µs - 57 inv. java.util.List.iterator
0.0% - 20 µs - 9 inv. java.util.LinkedList.<init>
0.0% - 8 µs - 9 inv. java.util.ArrayList.<init>(int)
0.0% - 8 µs - 53 inv. java.util.Queue.size
0.0% - 2 µs - 18 inv. java.util.List.size
1.0% - 35,975 µs - 10 inv. WildcardBFSSolution.ladderLength
1.0% - 33,683 µs - 10 inv. BidirectionalBFSSolution.ladderLength
0.8% - 29,281 µs - 1 inv. TestRunner.loadTestCases
0.6% - 21,514 µs - 35 inv. java.io.PrintStream.printf
0.0% - 194 µs - 15 inv. java.io.PrintStream.println(java.lang.String)
0.0% - 130 µs - 60 inv. java.util.Map.merge
0.0% - 110 µs - 10 inv. java.util.Map.entrySet
0.0% - 73 µs - 30 inv. java.util.ArrayList.<init>
0.0% - 65 µs - 12 inv. java.util.Set.iterator
0.0% - 52 µs - 36 inv. java.lang.Double.valueOf
0.0% - 46 µs - 59 inv. java.util.Iterator.hasNext
0.0% - 40 µs - 33 inv. java.lang.Long.valueOf
0.0% - 39 µs - 14 inv. java.lang.String.repeat

LIVE DEMO

Running on LeetCode Live Demo

Takeaways

1

BFS is the right lens

Words as nodes, one-letter swaps as edges → shortest path.

2

Wildcard buckets kill the N^2

Group once by pattern instead of comparing every pair.

3

Bidirectional search goes deeper

Two shallow trees beat one deep one on branchy graphs.

SOURCES

- LeetCode 127 · Word Ladder — leetcode.com/problems/word-ladder
- Bidirectional BFS: standard shortest-path optimization (CLRS, Introduction to Algorithms)